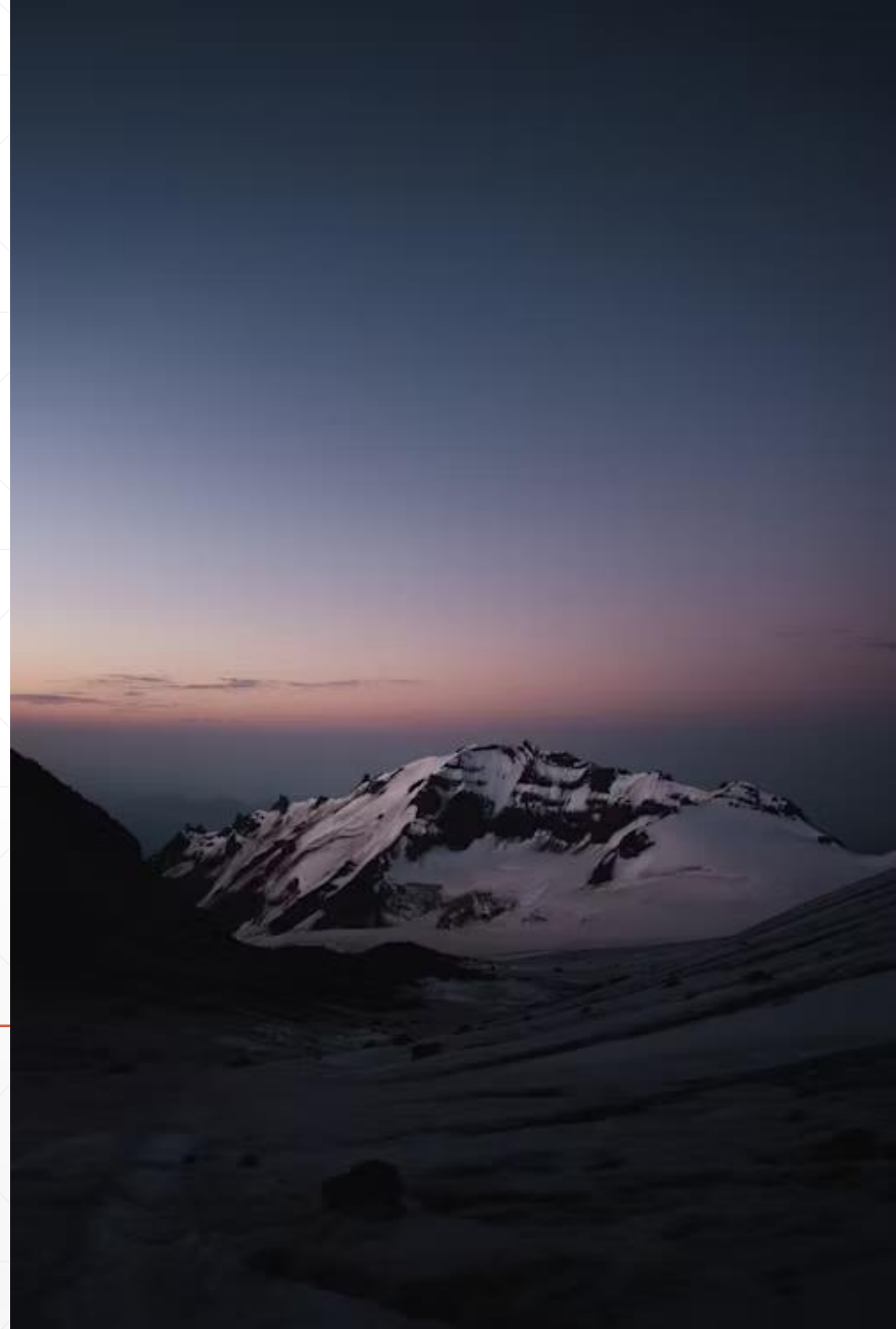


Netty初识（上）

开发服务端示例程序

北京理工大学计算机学院
金旭亮



概述



所有的网络应用，可以划分为两部分：客户端与服务器。



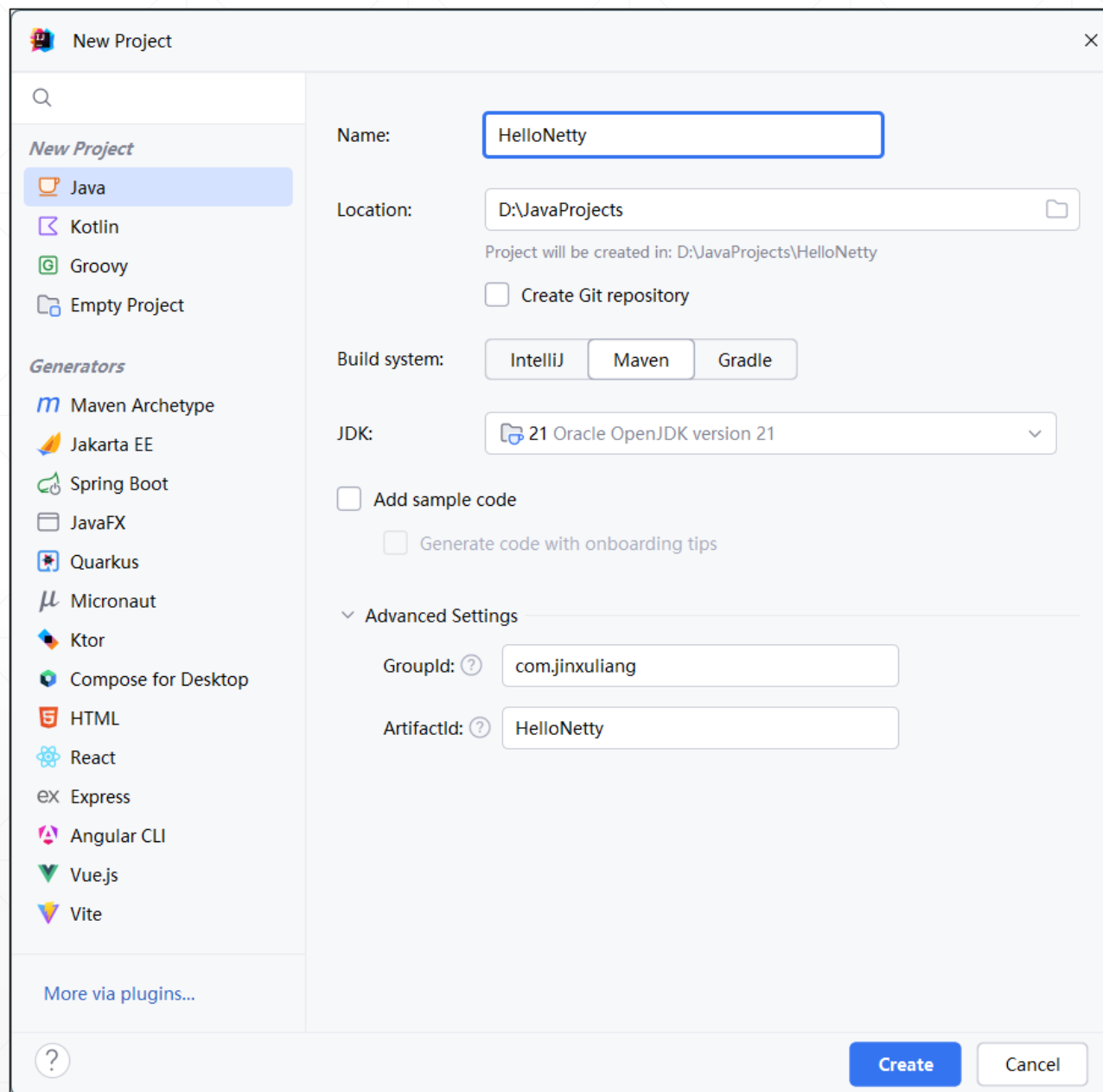
在本讲，我们将介绍一个最简单的Netty网络应用，基于TCP协议，完成以下最基础的功能——客户端连接服务端。具体步骤如下：

1. 服务端启动之后，等待客户端连接；
2. 客户端一启动，就立即连接服务端；
3. 服务端收到客户端连接请求，在控制台窗口输出信息；
4. 客户端断开连接。

创建项目

创建一个Maven项目，给其添加以下依赖：

```
<dependency>
  <groupId>io.netty</groupId>
  <artifactId>netty-all</artifactId>
  <version>4.1.112.Final</version>
</dependency>
```



通道与通道处理程序



在Netty中，客户端与服务端之间建立连接之后，数据通讯是由“通道（Channel）”来实现的。



依据底层使用的网络API，有多种类型的通道，比如，如果底层使用Java的NIO组件，位于服务端的通道，属于“NioServerSocketChannel”类型，而位于客户端的通道，则属于NioSocketChannel。



通道本身的基本职责，就是完成数据在计算机之间的传输，通常情况下，服务端程序还需要承担更多的职责，比如，它需要处理同一时刻到来的多个并发连接，往往需要有一个通道专门负责接收连接，然后，再为每个连接创建新的“子通道”，负责与对应的客户端进行数据交换。



当计算机收到数据之后，如何处理它们，这个任务，是由“通道处理程序（ChannelHandler）”承担的，ChannelHandler是Netty所定义的一个接口，是所有通道处理程序必须实现的接口。

ChannelHandler

  `exceptionCaught(ChannelHandlerContext, Throwable)` `void`

  `handlerAdded(ChannelHandlerContext)` `void`

  `handlerRemoved(ChannelHandlerContext)` `void`

ChannelInboundHandler

  `channelRegistered(ChannelHandlerContext)` `void`

  `channelActive(ChannelHandlerContext)` `void`

  `channelWritabilityChanged(ChannelHandlerContext)` `void`

  `channelInactive(ChannelHandlerContext)` `void`

  `channelRead(ChannelHandlerContext, Object)` `void`

  `channelUnregistered(ChannelHandlerContext)` `void`

  `channelReadComplete(ChannelHandlerContext)` `void`

  `userEventTriggered(ChannelHandlerContext, Object)` `void`

  `exceptionCaught(ChannelHandlerContext, Throwable)` `void`

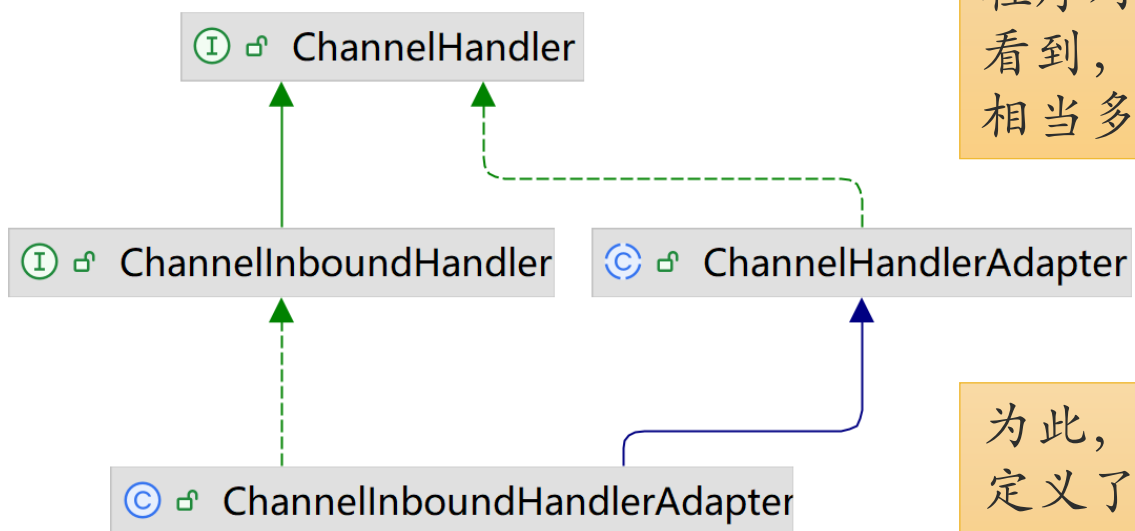
ChannelHandler（通道处理程序，简称“处理程序”）用于实现各种处理逻辑，分为两类，一类用于处理“进入（inbound）”本应用程序的消息，称为“入站消息”，需要实现ChannelInboundHandler接口，另一类用于处理“离开（outbound）”本应用程序的消息，称为“出站消息”，后面介绍。

多个ChannelHandler实例“首尾相接”，构成一个“Pipeline”，称为“流水线”或“管线”。可以动态地向“流水线”中添加入站处理程序或出站处理程序。

有关流水线的知识，后面介绍。

简化通道处理程序的编写

我们已经知道，程序收到外部传来的数据后，由通道处理程序对这些数据进行加工和处理。但在前页PPT中，我们看到，相关的接口，比如ChannelInboundHandler包容相当多的方法，一个一个地实现，是相当麻烦的。



为此，Netty引入了“适配器（Adapter）”设计模式，定义了一个ChannelInboundHandlerAdapter类型，在这里面，为相关接口所定义的方法，都提供了默认实现。

现在，工作简化为：从ChannelInboundHandlerAdapter中派生一个子类，仅仅只需重写感兴趣的方法即可。

编写服务端入站数据处理程序



当服务端接收到客户端连接请求时，需要输出一条信息，为此，编写了一个MyServerHandler，其代码如下所示：

```
//ChannelHandler类负责处理与客户端之间的数据交换
//
//Sharable注解表示，这个类的实例是Singleton的，可以被多个ChannelHandler流水线（pipeline）重用
//ChannelInboundHandlerAdapter是Netty内置的一个类，它实现了ChannelInboundHandler接口，
//为此接口中的方法提供了默认实现，我们可以选择需要的方法进行重写

@ChannelHandler.Sharable
public class MyServerHandler extends ChannelInboundHandlerAdapter {
    //当服务端与客户端的数据通道建立时，Netty会回调此方法
    @Override
    public void channelActive(ChannelHandlerContext ctx) throws Exception {
        System.out.println("有客户端连接：" + ctx.channel().remoteAddress());
    }
}
```



当有客户端连接进来时，Netty会“回调（callback）”channelActive方法，在这里，我们可以输出信息。注意方法参数，它是一个ChannelHandlerContext对象，这个对象包容有客户端的相关信息。

EventLoop与EventLoopGroup



网络应用程序，都是多线程的，服务端通常会有一个“无限循环”，等待客户端连接。在Netty中，Netty服务端应用，使用NioEventLoop来接收客户端连接请求和实现与客户端之间的数据双向通讯。



多个NioEventLoop聚成一组，称为“EventLoopGroup”，可以把它看成是Netty专门设计的线程池，能够高效地处理网络请求和传输数据。

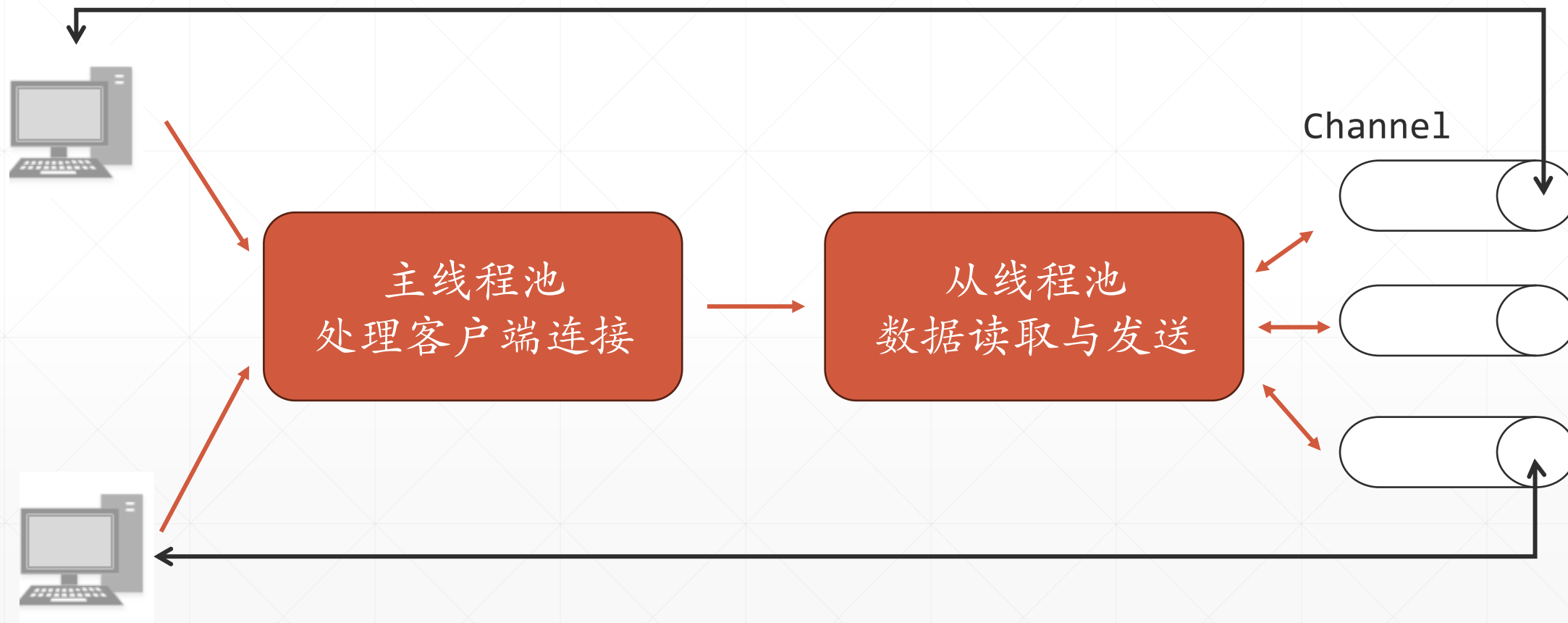


Netty服务端应用通常是双“EventLoopGroup”配置，一个负责接收客户端的网络连接请求，另一个则负责传输数据。



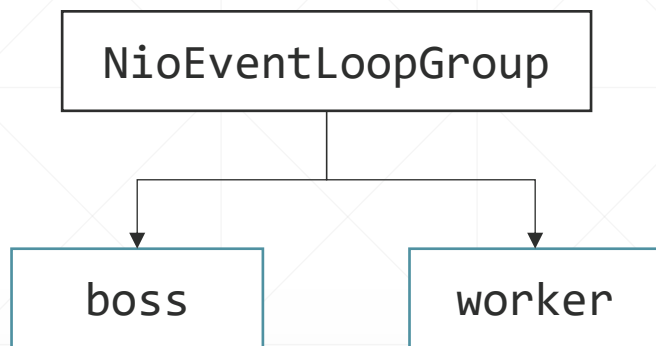
NioEventLoop中的线程，是非守护线程（daemon=false），不会主动退出，只有调用shutdown系列方法，才会退出。

Netty双线程池模型



服务器应用入口点

Netty推荐使用的“双线程组”模型



ServerBootstrap对象负责完成系统启动的所有工作。

```
public class MyNettyServer {
    private final int port = 9000;
    public void run() {
        EventLoopGroup bossGroup = new NioEventLoopGroup();
        EventLoopGroup workerGroup = new NioEventLoopGroup();
        try {
            ServerBootstrap boot = new ServerBootstrap();
            boot.group(bossGroup, workerGroup)
                .channel(NioServerSocketChannel.class)
                .childHandler(new MyServerHandler());
            ChannelFuture channelFuture = boot.bind(port).sync().addListener(nettyFuture -> {
                if (nettyFuture.isSuccess()) {
                    System.out.println(port + "端口绑定成功。");
                } else {
                    System.out.println(port + "端口绑定成功失败。");
                }
            });
            //如果服务端通道被关闭, 这里阻塞等待其完成
            channelFuture.channel().closeFuture().sync();
        } catch (Exception e) {
            System.out.println(e.getMessage());
        } finally {
            bossGroup.shutdownGracefully().addListener(future -> {
                System.out.println("bossGroup关闭。");
            });
            workerGroup.shutdownGracefully().addListener(future -> {
                System.out.println("workerGroup关闭。");
            });
        }
    }
    public static void main(String[] args) throws InterruptedException {
        new MyNettyServer().run();
    }
}
```

服务端代码解析

```
ServerBootstrap boot = new ServerBootstrap();  
boot.group(bossGroup, workerGroup)  
    .channel(NioServerSocketChannel.class)  
    .childHandler(new MyServerHandler());
```

上述代码中，实例化了一个ServerBootstrap对象，由它负责配置服务端应用：

首先，设置应用采用双EventLoopGroup方式，第一个bossGroup用于接收连接，第二个workGroup用于与Client通讯。

之后，设置应用使用NioServerSocketChannel通道完成数据传输任务。

第三，指定MyServerHandler实例负责实现数据加工与处理功能。

绑定端口



许多Netty组件，比如ServerBootstrap，它的bind()方法会返回一个ChannelFuture对象，调用它的addListener()，可以注册一个监听器对象，监听异步操作是否执行完成，调用ChannelFuture对象的sync()或await()方法，会阻塞当前调用线程，等待操作结束。

```
ChannelFuture channelFuture = boot.bind(port).sync().addListener(nettyFuture -> {  
    if (nettyFuture.isSuccess()) {  
        System.out.println(port + "端口绑定成功。");  
    } else {  
        System.out.println(port + "端口绑定成功失败。");  
    }  
});
```



以下这句代码的功能，其实是监听NioServerSocketChannel的关闭事件，阻塞当前调用线程，等待通道关闭（本示例需用户手动停止程序运行）。

```
//如果服务端通道被关闭，这里阻塞等待其完成  
channelFuture.channel().closeFuture().sync();
```

ChannelFuture接口

ChannelFuture是Netty定义的一个专用接口，用于实现异步操作。

Netty中许多组件的方法，都返回这一接口，如图所示，这一接口定义有很多方法，在后面的课程示例中，会经常看到这一接口的身影。

Future<V>		
resultNow()		V
exceptionNow()	Throwable	
state()	State	
get()		V
cancel(boolean)	boolean	
get(long, TimeUnit)		V
isCancelled()	boolean	
isDone()	boolean	

ChannelFuture		
await()		ChannelFuture
removeListener(GenericFutureListener<Future<Void>>)		ChannelFuture
removeListeners(GenericFutureListener<Future<Void>>[])		ChannelFuture
awaitUninterruptibly()		ChannelFuture
isVoid()	boolean	
channel()		Channel
addListener(GenericFutureListener<Future<Void>>)		ChannelFuture
addListeners(GenericFutureListener<Future<Void>>[])		ChannelFuture
syncUninterruptibly()		ChannelFuture
sync()		ChannelFuture

Future<V>		
cause()	Throwable	
addListeners(GenericFutureListener<Future<V>>[])		Future<V>
removeListeners(GenericFutureListener<Future<V>>[])		Future<V>
awaitUninterruptibly()		Future<V>
isSuccess()	boolean	
addListener(GenericFutureListener<Future<V>>)		Future<V>
await(long, TimeUnit)		boolean
removeListener(GenericFutureListener<Future<V>>)		Future<V>
awaitUninterruptibly(long)		boolean
awaitUninterruptibly(long, TimeUnit)		boolean
isCancellable()		boolean
sync()		Future<V>
syncUninterruptibly()		Future<V>
cancel(boolean)	boolean	
await(long)		boolean
await()		Future<V>
getNow()		V

程序关闭时的“收尾工作”

我们在前面说过，Netty的EventLoopGroup本质上是Netty定制的一个线程池，它并不会随着主线程的关闭而自行关闭，因此，程序中定义有一个finally语句块，当用户手动结束程序时，“优雅地”关闭这两个线程池。

```
finally {  
    bossGroup.shutdownGracefully().addListener(future -> {  
        System.out.println("bossGroup关闭。");  
    });  
    workerGroup.shutdownGracefully().addListener(future -> {  
        System.out.println("workerGroup关闭。");  
    });  
}
```

在finally语句块中关闭线程池，是Netty程序的典型作法。

注意一下shutdownGracefully()方法返回Future（如下所示），也是一个异步操作，示例给其添加了监听器对象，等其操作结束，输出相关信息。

```
Future<?> shutdownGracefully();
```

启动服务端程序

A screenshot of an IDE's Run console window. The title bar shows 'Run' and 'MyNettyServer'. The console output displays the Java command path and a success message.

```
Run MyNettyServer x
"C:\Program Files\Java\jdk-21\bin\java.exe" ...
9000端口绑定成功。
```

从输出中可以看到，现在服务端在9000端口监听，等待客户端的连接.....

小结：编写Netty服务器的步骤

所有的Netty服务器都需要以下两部分：

1

至少一个子ChannelHandler：该组件处理客户端传进来的数据，负责各种业务逻辑。

2

引导对象（Bootstrap）：这是配置服务器的启动代码。至少，它会将服务器绑定到它要监听连接请求的端口上。

Server端应用，主要由SeverBootstrap对象负责，它干的活主要是建立起整个服务端运行环境，创建好相应的线程池，通常是boss和worker两个线程池。程序启动之后，boss线程组负责监听连接请求，然后创建Channel，并负责组装好ChannelPipeline，后继的与Client端相互交换数据任务，主要就是由worker线程组中的线程操控Channel，调用pipeline中的各个ChannelHandler中的相应方法完成的。